

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
jhanasri@LAPTOP-1RN1M8E3: x + v
jhanasri@LAPTOP-1RN1M8E3:~$ mkdir OS_Skill_Week4
jhanasri@LAPTOP-1RN1M8E3:~$ cd OS_Skill_Week4
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ nano task1.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ls
task1.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ gcc task1.c -o task1
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ./task1
myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Parent PID: 611
Child PID: 612
Hello
Command execution completed.
myShell> echo Hello World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Argument 3: World
Parent PID: 611
Child PID: 615
Hello World
Command execution completed.
myShell> ls

--- Parsed Arguments ---
Argument 1: ls
Parent PID: 611
Child PID: 616
task1 task1.c
Command execution completed.
myShell>
Empty command.
myShell> |
```

tep 1: Open Ubuntu

Open **Ubuntu** from the Start menu.

You should see something like:

```
jhana@DESKTOP:~$
```

Step 2: Create a folder

Type:

```
mkdir OS_Skill_Week4
```

Press **Enter**.

Then:

```
cd OS_Skill_Week4
```

Your terminal should now show:

```
jhana@DESKTOP:~/OS_Skill_Week4$
```

Step 3: Create the C file

Type:

```
nano task1.c
```

Press **Enter**.

A blank editor will open.

Step 4: Paste the Task 1 code

Paste this:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 100
#define MAX_ARGS 20

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
```

```

while (1)
{
    // Display shell prompt
    printf("myShell> ");
    fflush(stdout);

    // Read input
    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        break;
    }

    // Remove newline character
    input[strcspn(input, "\n")] = '\0';

    // Check for empty command
    if (strlen(input) == 0)
    {
        printf("Empty command.\n");
        continue;
    }

    // Tokenization
    int count = 0;
    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count] = token;
        count++;

        token = strtok(NULL, " \t");
    }

    // End the argument list
    args[count] = NULL;

    // Display parsed arguments
    printf("\n--- Parsed Arguments ---\n");

    for (int i = 0; i < count; i++)
    {
        printf("Argument %d: %s\n", i + 1, args[i]);
    }

    // Create child process
    pid_t pid = fork();

```

```

if (pid < 0)
{
    perror("fork failed");
    continue;
}

// Child process
if (pid == 0)
{
    printf("Child PID: %d\n", getpid());

    // Execute command
    execvp(args[0], args);

    // Runs only if execvp fails
    perror("Command execution failed");
    exit(1);
}

// Parent process
else
{
    printf("Parent PID: %d\n", getpid());

    // Wait for child
    waitpid(pid, NULL, 0);

    printf("Command execution completed.\n");
}
}

return 0;
}

```

Step 5: Save the file

Inside **nano**:

Press:

Ctrl + O

Then press:

Enter

Then:

Ctrl + X

You will return to the Ubuntu terminal.

Step 6: Check the file

Type:

ls

You should see:

task1.c

Step 7: Compile

Type:

gcc task1.c -o task1

Press **Enter**.

Important

If nothing appears after this command, **that's good**. It means compilation succeeded.

Step 8: Run the program

Type:

./task1

You should see:

myShell>

Your shell program is running.

Step 9: Test **echo**

At:

myShell>

type:

echo Hello

Press Enter.

You should get something similar to:

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello

Child PID: 1234

Parent PID: 1200

Hello

Command execution completed.

Your PID numbers will be different.

The document's Task 1 example also demonstrates parsing `echo Hello`, followed by child-process execution and completion.

Step 10: Test multiple words

Type:

echo Hello World

You should get:

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello

Argument 3: World

Then the child executes:

Hello World

This demonstrates **tokenization**.

Step 11: Test `ls`

Type:

ls

You should see the files/folders in your current directory.

Your document's Task 5 example also uses `ls` to demonstrate child-process creation and execution.

Step 12: Test empty command

At:

myShell>

just press **Enter**.

You should get:

Empty command.

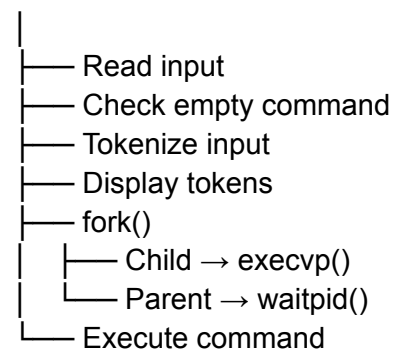
Then the shell returns to:

myShell>

This satisfies the **handle empty commands** requirement from Task 1.

What you have completed

TASK 1



Your commands, in order

mkdir OS_Skill_Week4

```
cd OS_Skill_Week4  
nano task1.c
```

Paste code → **Ctrl+O** → **Enter** → **Ctrl+X**

Then:

```
gcc task1.c -o task1  
./task1
```

Then test:

```
echo Hello  
echo Hello World  
ls
```

=====

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
jhanasri@LAPTOP-1RN1M8E3: ~$ cd ~/OS_Skill_Week4
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ls
task1 task1.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ nano task2.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ gcc task2.c -o task2
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ./task2
myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Child PID: 780
Hello
Command execution completed.
myShell> echo 'Hello World'

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Child PID: 783
Hello World
Command execution completed.
myShell> Hello World

--- Parsed Arguments ---
Argument 1: Hello
Argument 2: World
Child PID: 784
Command execution failed: No such file or directory
Command execution completed.
myShell> echo 'Hello World'
Syntax Error: Unmatched single quote.
myShell> echo '$USER'

--- Parsed Arguments ---
Argument 1: echo
Argument 2: $USER
Child PID: 785
$USER
Command execution completed.
myShell> |
```

Step 1: Go to your Task 1 folder

Open Ubuntu and type:

```
cd ~/OS_Skill_Week4
```

Check:

```
ls
```

You should see:

task1
task1.c

Step 2: Create Task 2 file

Type:

nano task2.c

Step 3: Paste this code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);

        // Read input
        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Check empty command
        if (strlen(input) == 0)
        {
            printf("Empty command.\n");
            continue;
        }
    }
}
```

```

// Check for unmatched single quote
int singleQuote = 0;

for (int i = 0; input[i] != '\0'; i++)
{
    if (input[i] == '"')
    {
        singleQuote = !singleQuote;
    }
}

if (singleQuote)
{
    printf("Syntax Error: Unmatched single quote.\n");
    continue;
}

// Tokenization
int count = 0;
int inSingleQuote = 0;
char token[200];
int tokenIndex = 0;

for (int i = 0; ; i++)
{
    char ch = input[i];

    // Start/end single quote
    if (ch == '"')
    {
        inSingleQuote = !inSingleQuote;
        continue;
    }

    // Space or end of input
    if ((ch == ' ' || ch == '\t' || ch == '\0')
        && !inSingleQuote)
    {
        if (tokenIndex > 0)
        {
            token[tokenIndex] = '\0';

            args[count] = malloc(strlen(token) + 1);
            strcpy(args[count], token);

            count++;
            tokenIndex = 0;
        }
    }
}

```

```

    }

    if (ch == '\0')
        break;

    continue;
}

// Store character
token[tokenIndex++] = ch;
}

args[count] = NULL;

// Display parsed arguments
printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < count; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}

// Create child process
pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
}

// Child process
else if (pid == 0)
{
    printf("Child PID: %d\n", getpid());

    execvp(args[0], args);

    perror("Command execution failed");
    exit(1);
}

// Parent process
else
{
    waitpid(pid, NULL, 0);
    printf("Command execution completed.\n");
}

```

```
        // Free allocated memory
        for (int i = 0; i < count; i++)
        {
            free(args[i]);
        }
    }

    return 0;
}
```

Step 4: Save

Inside **nano**:

Ctrl + O

Press:

Enter

Then:

Ctrl + X

Step 5: Compile

Run:

```
gcc task2.c -o task2
```

If there is **no error**, compilation succeeded.

Step 6: Run

```
./task2
```

You should see:

```
myShell>
```

Step 7: Test normal **echo**

Enter:

echo Hello

Expected:

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello

Hello

Step 8: Test single quotes

Now enter:

echo 'Hello World'

The important point is that **the space inside the single quotes is preserved**.

Expected parsing:

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello World

Then:

Hello World

This is the behavior required by Task 2: quoted content should be preserved as a single argument.

Step 9: Test unmatched single quote

Enter:

echo 'Hello World

There is no closing ' .

Your program should display:

Syntax Error: Unmatched single quote.

This exactly matches the **Case 2** behavior in your document.

Step 10: Test literal \$USER

This is important for understanding single quotes.

Enter:

```
echo '$USER'
```

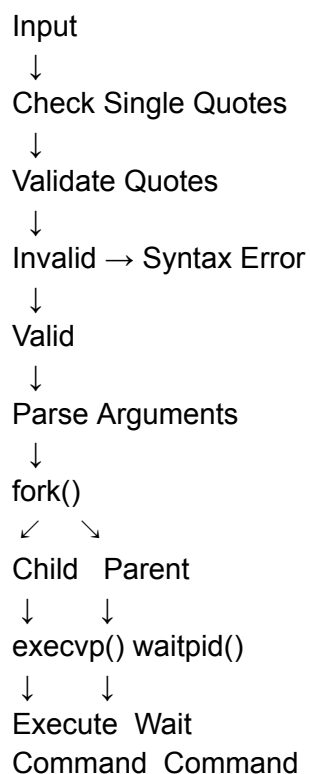
It should print:

```
$USER
```

The `$USER` should **not be expanded** because it is inside single quotes.

flow

Your document's flowchart essentially follows:



Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
jhanasri@LAPTOP-1RN1M8E3: X + v
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OS_Skill_Week4
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ls
task1 task1.c task2 task2.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ nano task3.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ gcc task3.c -o task3
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ./task3
myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Child Process
PID = 987
Hello
Child process completed.
myShell> echo "Hello World"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Child Process
PID = 988
Hello World
Child process completed.
myShell> echo "Hello $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello jhanasri
Child Process
PID = 989
Hello jhanasri
Child process completed.
myShell> echo "Hello World
Syntax Error: Unmatched double quote.
myShell> |
```

Step 1: Go to your folder

Open Ubuntu:

```
cd ~/OS_Skill_Week4
```

Check:

```
ls
```


You should have:

```
task1
task1.c
task2
task2.c
```

Step 2: Create Task 3

```
nano task3.c
```

Step 3: Paste this complete code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);

        // Read command
        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Check empty command
        if (strlen(input) == 0)
        {
            printf("Empty command.\n");
            continue;
        }
    }
}
```

```

}

// Check for unmatched double quote
int doubleQuote = 0;

for (int i = 0; input[i] != '\0'; i++)
{
    if (input[i] == '"')
    {
        doubleQuote = !doubleQuote;
    }
}

if (doubleQuote)
{
    printf("Syntax Error: Unmatched double quote.\n");
    continue;
}

// Tokenization
int count = 0;
int inDoubleQuote = 0;
char token[MAX_INPUT];
int tokenIndex = 0;

for (int i = 0; ; i++)
{
    char ch = input[i];

    // Handle double quote
    if (ch == '"')
    {
        inDoubleQuote = !inDoubleQuote;
        continue;
    }

    // Handle space or end of input
    if ((ch == ' ' || ch == '\t' || ch == '\0')
        && !inDoubleQuote)
    {
        if (tokenIndex > 0)
        {
            token[tokenIndex] = '\0';

            args[count] = malloc(strlen(token) + 1);
            strcpy(args[count], token);

            count++;
        }
    }
}

```

```

        tokenIndex = 0;
    }

    if (ch == '\0')
    {
        break;
    }

    continue;
}

// Handle $USER variable
if (ch == '$')
{
    if (strncmp(&input[i], "$USER", 5) == 0)
    {
        char *user = getenv("USER");

        if (user != NULL)
        {
            strcpy(&token[tokenIndex], user);
            tokenIndex += strlen(user);
        }

        i += 4;
        continue;
    }
}

// Store character
token[tokenIndex++] = ch;
}

// End arguments
args[count] = NULL;

// Display parsed arguments
printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < count; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}

// Create child process
pid_t pid = fork();

if (pid < 0)

```

```

    {
        perror("fork failed");
        continue;
    }

    // Child process
    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());

        execvp(args[0], args);

        perror("Command execution failed");
        exit(1);
    }

    // Parent process
    else
    {
        waitpid(pid, NULL, 0);
        printf("Child process completed.\n");
    }

    // Free memory
    for (int i = 0; i < count; i++)
    {
        free(args[i]);
    }
}

return 0;
}

```

Step 4: Save the file

In **nano**:

Ctrl + O

Press **Enter**.

Then:

Ctrl + X

Step 5: Compile

```
gcc task3.c -o task3
```

If there is **no error**, compilation is successful.

Step 6: Run

```
./task3
```

You should see:

```
myShell>
```

Step 7: Test normal command

Enter:

```
echo Hello
```

Expected:

```
--- Parsed Arguments ---
```

```
Argument 1: echo
```

```
Argument 2: Hello
```

Step 8: Test double quotes

Enter:

```
echo "Hello World"
```

Expected:

```
--- Parsed Arguments ---
```

```
Argument 1: echo
```

```
Argument 2: Hello World
```

Notice that **Hello World becomes one argument** even though there is a space.

Step 9: Test **\$USER**

Enter:

```
echo "Hello $USER"
```

Your document gives this exact test case.

You should get something like:

```
--- Parsed Arguments ---
```

```
Argument 1: echo
```

```
Argument 2: Hello jhana
```

```
Child Process
```

```
PID = 3022
```

```
Hello jhana
```

```
Child process completed.
```

The username will be **your Ubuntu username**, so it may not be `jhana`. The document's sample shows `Hello raghu`.

Step 10: Test unmatched double quote

Enter:

```
echo "Hello World
```

Because the closing `"` is missing, you should get:

```
Syntax Error: Unmatched double quote.
```

This is exactly the **Case 2** requirement in your document.

flow

Input



Check Double Quotes



Validate Quotes



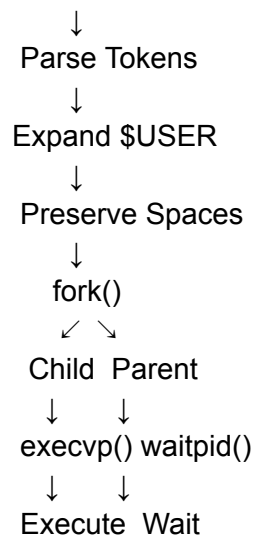
Unmatched?



Yes → Syntax Error



No



Task 4:

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
jhanasri@LAPTOP-1RN1M8E3: x + v
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OS_Skill_Week4
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ls
task1 task1.c task2 task2.c task3 task3.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ nano task4.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ gcc task4.c -o task4
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ./task4
myShell> echo Hello

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello
Child PID: 1341
Hello
Command execution completed.
myShell> echo Hello\ World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Child PID: 1342
Hello World
Command execution completed.
myShell> Hello World

--- Parsed Arguments ---
Argument 1: Hello
Argument 2: World
Child PID: 1345
Command execution failed: No such file or directory
Command execution completed.
myShell> echo Hello\!

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello!
Child PID: 1348
Hello!
Command execution completed.
myShell> echo Hello\\World
```



```
Command execution completed.
myShell> Hello World

--- Parsed Arguments ---
Argument 1: Hello
Argument 2: World
Child PID: 1345
Command execution failed: No such file or directory
Command execution completed.
myShell> echo Hello\!

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello!
Child PID: 1348
Hello!
Command execution completed.
myShell> echo Hello\\World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello\World
Child PID: 1349
Hello\World
Command execution completed.
myShell> echo This\ is\ a\ test

--- Parsed Arguments ---
Argument 1: echo
Argument 2: This is a test
Child PID: 1352
This is a test
Command execution completed.
myShell> echo "Hello\ World"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Child PID: 1353
Hello World
Command execution completed.
```

Step 1 — Go to your folder

`cd ~/OS_Skill_Week4`

Check your files:

`ls`

You should have something like:

```
task1.c
task1
task2.c
task2
task3.c
task3
```

Step 2 — Create Task 4

nano task4.c

Step 3 — Paste the complete code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1)
    {
        printf("myShell> ");
        fflush(stdout);

        // Read input
        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Check empty command
        if (strlen(input) == 0)
        {
            printf("Empty command.\n");
            continue;
        }

        int count = 0;
        char token[MAX_INPUT];
        int tokenIndex = 0;

        int escaped = 0;
```

```

int singleQuote = 0;
int doubleQuote = 0;

for (int i = 0; ; i++)
{
    char ch = input[i];

    // End of input
    if (ch == '\0')
    {
        if (escaped)
        {
            printf("Syntax Error: Incomplete escape sequence.\n");
            break;
        }

        if (singleQuote || doubleQuote)
        {
            printf("Syntax Error: Unmatched quote.\n");
            break;
        }

        // Store last token
        if (tokenIndex > 0)
        {
            token[tokenIndex] = '\0';

            args[count] = malloc(strlen(token) + 1);
            strcpy(args[count], token);

            count++;
            tokenIndex = 0;
        }

        break;
    }

    // Handle escape character
    if (ch == '\\' && !singleQuote)
    {
        escaped = !escaped;

        if (escaped)
        {
            continue;
        }
    }
}

```

```

// If previous character was backslash,
// preserve the current character
if (escaped)
{
    token[tokenIndex++] = ch;
    escaped = 0;
    continue;
}

// Handle single quote
if (ch == "\"" && !doubleQuote)
{
    singleQuote = !singleQuote;
    continue;
}

// Handle double quote
if (ch == "'" && !singleQuote)
{
    doubleQuote = !doubleQuote;
    continue;
}

// Handle spaces
if ((ch == ' ' || ch == '\t')
    && !singleQuote && !doubleQuote)
{
    if (tokenIndex > 0)
    {
        token[tokenIndex] = '\0';

        args[count] = malloc(strlen(token) + 1);
        strcpy(args[count], token);

        count++;
        tokenIndex = 0;
    }

    continue;
}

// Store normal character
token[tokenIndex++] = ch;
}

// If syntax error occurred
if (singleQuote || doubleQuote || escaped)
{

```

```

    for (int i = 0; i < count; i++)
    {
        free(args[i]);
    }

    continue;
}

args[count] = NULL;

// No command
if (count == 0)
{
    printf("Empty command.\n");
    continue;
}

// Display parsed arguments
printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < count; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}

// Create child process
pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
}
else if (pid == 0)
{
    // Child process
    printf("Child PID: %d\n", getpid());

    execvp(args[0], args);

    perror("Command execution failed");
    exit(1);
}
else
{
    // Parent process
    waitpid(pid, NULL, 0);

    printf("Command execution completed.\n");
}

```

```
    }

    // Free memory
    for (int i = 0; i < count; i++)
    {
        free(args[i]);
    }
}

return 0;
}
```

Step 4 — Save

In nano:

Ctrl + O

Press **Enter**.

Then:

Ctrl + X

Step 5 — Compile

```
gcc task4.c -o task4
```

If Ubuntu shows **no error**, compilation succeeded.

Step 6 — Run

```
./task4
```

You should see:

```
myShell>
```

Step 7 — Test normal command

Enter:

echo Hello

Expected:

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello

Step 8 — Test escaped space

Enter:

echo Hello\ World

The `\` tells the parser that the space should be treated as part of the argument.

Expected:

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello World

Output:

Hello World

Step 9 — Test escaped special character

Try:

echo Hello\!

Expected:

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello!

The backslash preserves the `!` character.

Step 10 — Test escaped backslash

Try:

```
echo Hello\\World
```

Expected:

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello\World

Step 11 — Test multiple escaped spaces

Try:

```
echo This\ is\ a\ test
```

Expected:

--- Parsed Arguments ---

Argument 1: echo

Argument 2: This is a test

Notice that the entire sentence is treated as **one argument**.

Step 12 — Test a quoted command with an escaped character

Try:

```
echo "Hello\ World"
```

The parser preserves the escaped character while keeping the quoted text together.

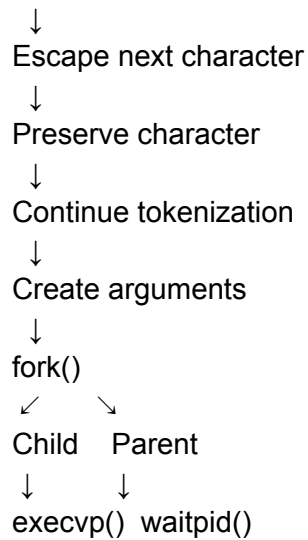
concept

The important part is:

Input

↓

Check '\'



Task 5:

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
jhanasri@LAPTOP-1RN1M8E3:~$ cd ~/OS_Skill_Week4
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ls
task1 task1.c task2 task2.c task3 task3.c task4 task4.c task5 task5.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ nano task5.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ls
task1 task1.c task2 task2.c task3 task3.c task4 task4.c task5 task5.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ gcc task5.c -o task5
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ls
task1 task1.c task2 task2.c task3 task3.c task4 task4.c task5 task5.c
jhanasri@LAPTOP-1RN1M8E3:~/OS_Skill_Week4$ ./task5
Parent Process PID: 1602
Created Child PID: 1603
Child Process PID: 1603
task1 task1.c task2 task2.c task3 task3.c task4 task4.c task5 task5.c
Child exited with status: 0
```

Step 1 — Open Ubuntu

Open **Ubuntu**.

Step 2 — Go to your folder

Type:

```
cd ~/OS_Skill_Week4
```

Press **Enter**.

Step 3 — Check the folder

Type:

```
ls
```

You should see your previous files:

```
task1
task1.c
task2
task2.c
task3
task3.c
task4
task4.c
```

Step 4 — Create Task 5

Type:

```
nano task5.c
```

Press **Enter**.

Step 5 — Paste this code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    printf("Parent Process PID: %d\n", getpid());
```

```

pid_t pid = fork();

if (pid < 0)
{
    perror("fork failed");
    return 1;
}

else if (pid == 0)
{
    printf("Created Child PID: %d\n", getpid());
    printf("Child Process PID: %d\n", getpid());

    char *args[] = {"ls", NULL};

    execvp(args[0], args);

    perror("execvp failed");
    exit(1);
}

else
{
    int status;

    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child exited with status: %d\n",
            WEXITSTATUS(status));
    }
    else
    {
        printf("Child did not exit normally.\n");
    }
}

return 0;
}

```

Step 6 — Save

Press:

Ctrl + O

Then press:

Enter

Step 7 — Exit nano

Press:

Ctrl + X

You are back at the Ubuntu terminal.

Step 8 — Check the file

Type:

ls

You should now see:

task5.c

Step 9 — Compile

Type:

gcc task5.c -o task5

Press **Enter**.

If there is **no error**, compilation succeeded

Step 10 — Check executable

Type:

ls

Now you should see:

task5
task5.c

Step 11 — Run

Type:

```
./task5
```

Step 12 — Expected output

You should get something similar to:

Parent Process PID: 1470

Created Child PID: 1471

Child Process PID: 1471

task1 task1.c task2 task2.c task3 task3.c task4 task4.c task5 task5.c

Child exited with status: 0

Your PID numbers will be different.

This is the same type of output shown in your document for Task 5.

Now understand the code

Step 13 — `fork()`

This line:

```
pid_t pid = fork();
```

creates a **child process**.

```
      fork()
     /    \
    /      \
  Parent   Child
```

Step 14 — Check `fork()`

```
if (pid < 0)
```

Means:

fork failed.

Step 15 — Child

```
else if (pid == 0)
```

Means:

this is the child process.

Step 16 — Create arguments

```
char *args[] = {"ls", NULL};
```

This tells the child that we want to execute:

```
ls
```

Step 17 — Execute command

```
execvp(args[0], args);
```

This runs the `ls` command.

That's why you see:

```
task1
task1.c
task2
task2.c
...
```

Step 18 — Parent

The `else` part belongs to the parent:

```
else
{
    ...
}
```

Step 19 — Parent waits

```
waitpid(pid, &status, 0);
```

The parent waits until the child finishes.

Step 20 — Check result

WEXITSTATUS(status)

gets the child's exit status.

If you see:

Child exited with status: 0

it means **successful execution**

Step 21 — Test error handling

Now we test the error part required by Task 5.

Open the file:

```
nano task5.c
```

Step 22 — Find this line

```
char *args[] = {"ls", NULL};
```

Change it to:

```
char *args[] = {"wrongcommand", NULL};
```

Step 23 — Save

Ctrl + O

Press **Enter**.

Then:

Ctrl +

Step 24 — Compile again

```
gcc task5.c -o task5
```

Step 25 — Run

```
./task5
```

You should see something like:

Parent Process PID: 1500
Created Child PID: 1501
Child Process PID: 1501
execvp failed: No such file or directory
Child exited with status: 1

This tests the execution-error handling.

Step 26 — Put it back to normal

Open:

nano task5.c

Change:

```
char *args[] = {"wrongcommand", NULL};
```

back to:

```
char *args[] = {"ls", NULL};
```

Step 27 — Save

Ctrl + O

Enter

Ctrl + X

Step 28 — Compile final version

```
gcc task5.c -o task5
```

Step 29 — Run final version

```
./task5
```

You should get:

Parent Process PID: XXXX

Created Child PID: XXXX

Child Process PID: XXXX

task1 task1.c task2 task2.c task3 task3.c task4 task4.c task5 task5.c

Child exited with status: 0